

The mathematics of Beta-Yee diagrams

Exact ranking probabilities of Beta and normal voters, and how the two models differ

Contents

1. Generating the ranking probabilities	2
1.1. Problem	2
1.2. Voter distribution of a pixel	2
1.2.1. Model	2
1.2.2. The two equations (<code>_solve_beta</code>)	2
1.2.3. Symmetry and continuation (<code>beta_params_at</code>)	3
1.3. Ranking cells (<code>ranking_cells</code>)	3
1.3.1. Bisectors	3
1.3.2. Construction	3
1.4. Cell probability as a boundary integral	4
1.4.1. Green's theorem	4
1.4.2. Sharing edges (<code>compute_ranking_probabilities</code>)	5
1.5. Edge integrals (<code>_edge_integral</code>)	5
1.5.1. Axis-parallel edges (closed form)	5
1.5.2. General edges: substitution $u = F(x)$	5
1.5.3. Edges touching $y = 0$ or $y = 1$: substitution $v = G(y)$	5
1.5.4. Edges touching both kinds of boundary	5
1.5.5. Quadrature	5
1.6. Interpolation in the pixel median	6
1.6.1. Nodes (<code>node_medians</code>)	6
1.6.2. Barycentric interpolation (<code>_interpolation_matrix</code>)	6
1.7. Validation (<code>monte_carlo_check</code>)	6
1.8. Summary of the pipeline	7
1.9. Normal voters (<code>normal.py</code>)	7
1.9.1. Model	7
1.9.2. Cells in a larger box (<code>normal_cells</code>)	7
1.9.3. Exact cell probabilities with Owen's T function	7
1.9.4. Nodes	8
2. Beta versus normal voters	9
2.1. The two models	9
2.2. Pairwise majorities	10
2.2.1. Normal voters: the majority is decided by distance	10
2.2.2. Beta voters: the median is a median only along the axes	10
2.2.3. How skewed the Beta is at $d = 0.3$	10
2.2.4. Example: the pull towards the centre	11
2.2.5. The whole diagram	11
2.3. Round edges	11
2.3.1. First-choice shares are blurred Voronoi cells	11
2.3.2. Straight versus round: a proof by blurring a corner	11
2.3.3. Seven candidates	12
2.3.4. The round IRV edge	13
2.3.5. Hooks at the edge of the square (FPTP)	15
2.4. Summary	16

1. Generating the ranking probabilities

1.1. Problem

The political space is the unit square $[0, 1]^2$. There are C candidates at fixed points $c_1, \dots, c_C \in [0, 1]^2$. The square is divided into $n \times n$ pixels (PIXELS), and each pixel represents a population of voters, not a single voter.

A voter at position p ranks the candidates by Euclidean distance: the closest candidate is ranked first, the next closest second, and so on. For each pixel we want

$$P_{kl}(r) = \text{share of the voters of pixel } (k, l) \text{ whose ranking is } r \quad (1)$$

for every strict ranking r that can occur. These numbers feed the voting methods (Borda, Schulze, ...) in `methods.py`. The module computes them **without sampling or discretising the voters**: the only approximations are Gauss–Legendre quadrature and polynomial interpolation, both of which converge exponentially fast.

1.2. Voter distribution of a pixel

1.2.1. Model

The voters of pixel (k, l) have positions (X, Y) with X and Y independent,

$$X \sim \text{Beta}(a_k, b_k), \quad Y \sim \text{Beta}(a_l, b_l). \quad (2)$$

The Beta family is used because it lives on $[0, 1]$, so every voter stays inside the political space. Parameters (a, b) are chosen per coordinate so that

1. the **median** is the pixel centre $m = (k + \frac{1}{2})/n$ (`pixel_medians`), and
2. the **mean absolute deviation from the median** is a fixed constant d (`DEVIATION`): $E|X - m| = d$.

Condition 2 keeps the “spread” of every pixel the same, independently of where the pixel lies. Near the edges of the square this forces $a < 1$ or $b < 1$, so the density is singular (infinite) at 0 or 1; much of the later numerics is shaped by this.

We write f, F for the pdf and CDF of X and g, G for those of Y . The CDF of $\text{Beta}(a, b)$ is the regularised incomplete beta function $I_x(a, b)$ (`scipy.special.betainc`), its inverse is `betaincinv`.

1.2.2. The two equations (`_solve_beta`)

The median condition is simply

$$I_m(a, b) = \frac{1}{2}. \quad (3)$$

For the absolute deviation, split at the median and use $F(m) = \frac{1}{2}$:

$$\begin{aligned} E|X - m| &= E[X - m] - 2E[(X - m)\mathbb{1}\{X < m\}] \\ &= \mu - m - 2E[X\mathbb{1}\{X < m\}] + 2mF(m) \\ &= \mu - 2E[X\mathbb{1}\{X < m\}], \end{aligned} \quad (4)$$

where $\mu = \frac{a}{a+b}$. Since $x \cdot x^{a-1}(1-x)^{b-1}/B(a, b) = \mu \cdot (\text{density of Beta}(a+1, b))$,

$$E[X\mathbb{1}\{X < m\}] = \mu I_m(a+1, b), \quad (5)$$

and therefore

$$E|X - m| = \frac{a}{a+b}(1 - 2I_m(a+1, b)) = d. \quad (6)$$

Equation 3 and Equation 6 are two equations in the two unknowns (a, b) and are solved with Levenberg–Marquardt (`scipy.optimize.root(method="lm")`).

1.2.3. Symmetry and continuation (beta_params_at)

If $X \sim \text{Beta}(a, b)$ then $1 - X \sim \text{Beta}(b, a)$. A median $m < \frac{1}{2}$ is therefore the mirror of median $1 - m$ with (a, b) swapped, so only the medians $\max(m, 1 - m) \geq \frac{1}{2}$ are solved. They are solved in increasing order, starting from $(1, 1)$ (the uniform distribution, median $\frac{1}{2}$) and using each solution as the initial guess for the next median. This continuation keeps the solver on the correct branch even far from the centre, where (a, b) change quickly.

1.3. Ranking cells (ranking_cells)

1.3.1. Bisectors

A voter at p prefers c_i to c_j exactly when

$$|p - c_i|^2 < |p - c_j|^2 \iff 2(c_j - c_i) \cdot p < |c_j|^2 - |c_i|^2. \quad (7)$$

The boundary of Equation 7 is the perpendicular bisector of $c_i c_j$, a straight line. A voter's full ranking is determined by the signs of all $\binom{C}{2}$ comparisons, so the ranking can only change when the voter crosses one of these bisectors.

The $\binom{C}{2}$ lines cut the unit square into convex polygons ("cells"). Inside a cell every comparison has a fixed sign, hence **every point of a cell has the same ranking**. Points on a bisector have a tie, but lines have zero area and so zero probability, and ties are ignored.

The crucial observation is that **the cells do not depend on the pixel**: they depend only on the candidates. Only the probability mass that each pixel's distribution puts on each cell changes. So

$$P_{kl}(r) = P_{kl}((X, Y) \in K_r) = \iint_{K_r} f_k(x)g_l(y) \, dx \, dy, \quad (8)$$

where K_r is the cell with ranking r .

1.3.2. Construction

The construction starts with the square as a single polygon and, for every pair (i, j) , splits every current polygon into its parts on either side of the bisector (half-plane clipping, `_clip`, a Sutherland–Hodgman step for one line). Slivers with area below 10^{-14} are discarded. All polygons stay convex and counter-clockwise (CCW). The ranking of a cell is found by sorting the candidates by distance from the cell's vertex average, which lies strictly inside the convex cell.

Figure 1 shows the arrangement for the candidates in `const.py`.

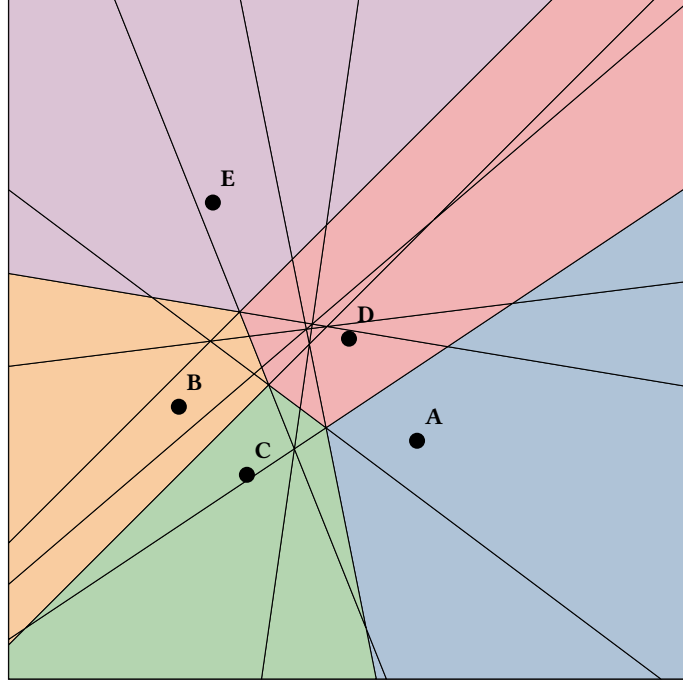


Figure 1: The 43 ranking cells of the default candidates A–E (rows of CANDIDATES). Colour marks the first choice; every cell has its own full ranking.

1.4. Cell probability as a boundary integral

Computing the double integral Equation 8 directly for every cell and every pixel pair would be expensive. Green's theorem turns it into a sum of integrals along the cell's edges, and each edge is shared by two cells.

1.4.1. Green's theorem

For a region K with CCW boundary ∂K and a 1-form $\omega = L dx + M dy$,

$$\oint_{\partial K} L dx + M dy = \iint_K (\partial_x M - \partial_y L) dx dy. \quad (9)$$

Choose

$$\omega = -f(x)G(y) dx. \quad (10)$$

Then $\partial_x M - \partial_y L = f(x)g(y)$, which is the joint density, so

$$P(K) = \oint_{\partial K} \omega = \sum_{\text{edges } s \rightarrow e} \int_s^e \omega. \quad (11)$$

An equally valid choice is

$$\omega' = F(x)g(y) dy, \quad \omega' - \omega = Fg dy + fG dx = d(FG). \quad (12)$$

Because ω' and ω differ by an exact form, on any segment

$$\int_s^e \omega = \int_s^e \omega' - [F(x)G(y)]_s^e. \quad (13)$$

Which form is integrated numerically is chosen edge by edge (Section 1.5).

Sanity check. The probabilities of all cells sum to one: interior edges are traversed once in each direction and cancel, leaving the boundary of the square. On it $dx = 0$ on the vertical sides, $G(0) = 0$ on the bottom, and the top ($y = 1$, traversed from $x = 1$ to $x = 0$) gives $-\int_1^0 f(x) dx = 1$.

1.4.2. Sharing edges (compute_ranking_probabilities)

Every edge is stored once under a canonical orientation (lexicographically smaller endpoint first). A cell that traverses the edge in the canonical direction adds its integral with sign +1; the neighbouring cell traverses it the other way and adds it with sign −1. The edges are integrated in parallel threads (betainc / betaincinv release the GIL).

The result of each edge integral is a matrix: entry $[k, l]$ uses the X -parameters of pixel column k and the Y -parameters of pixel row l . Because X and Y are independent, one set of n parameter pairs serves both axes, and the whole $n \times n$ grid of pixels comes out of $O(n)$ one-dimensional quantities combined in an outer-product fashion.

1.5. Edge integrals (_edge_integral)

Let the edge go from $s = (x_s, y_s)$ to $e = (x_e, y_e)$.

1.5.1. Axis-parallel edges (closed form)

- **Vertical** ($x_s = x_e$): $dx = 0$, so $\int \omega = 0$.
- **Horizontal** ($y_s = y_e$): $G(y)$ is constant, so

$$\int_s^e \omega = -G(y_s)(F(x_e) - F(x_s)). \quad (14)$$

These include all edges on the boundary of the square.

1.5.2. General edges: substitution $u = F(x)$

On the line through s and e , $y = \ell(x) = y_s + (x - x_s)(y_e - y_s)/(x_e - x_s)$. Integrating Equation 10 directly is a bad idea: $f(x)$ is infinite at 0 or 1 when $a < 1$ or $b < 1$, and quadrature would converge slowly. Substituting $u = F(x)$, $du = f(x) dx$ absorbs the density:

$$\int_s^e \omega = - \int_{F(x_s)}^{F(x_e)} G(\ell(F^{-1}(u))) du. \quad (15)$$

The integrand is now bounded by 1 and smooth unless the edge reaches $y = 0$ or $y = 1$, where G itself behaves like y^a or $1 - (1 - y)^b$ and is not smooth.

1.5.3. Edges touching $y = 0$ or $y = 1$: substitution $v = G(y)$

For those edges the roles of the axes are swapped. Integrate ω' from Equation 12 with $v = G(y)$, $dv = g(y) dy$, and $x = \ell^{-1}(y)$:

$$\int_s^e \omega' = \int_{G(y_s)}^{G(y_e)} F(\ell^{-1}(G^{-1}(v))) dv, \quad (16)$$

and convert back to ω with Equation 13. Now the endpoint on $y \in \{0, 1\}$ is harmless because it is mapped to $v \in \{0, 1\}$ and the singular behaviour of G is absorbed by the substitution, while x stays in the interior of $(0, 1)$ so F is smooth there.

1.5.4. Edges touching both kinds of boundary

An edge can run from a vertical side of the square ($x \in \{0, 1\}$) to a horizontal one ($y \in \{0, 1\}$). Neither substitution handles both endpoints, so the edge is split at its midpoint; each half touches only one kind of boundary and is handled as above. (An edge ending exactly at a corner of the square is integrated with the v form.)

1.5.5. Quadrature

The remaining one-dimensional integrals over $[u_s, u_e]$ (or $[v_s, v_e]$) are computed with Q -point Gauss–Legendre quadrature (QUAD_NODES, default $Q = 24$):

$$\int_{u_s}^{u_e} h(u) du \approx \frac{u_e - u_s}{2} \sum_{q=1}^Q w_q h\left(u_s + (u_e - u_s) \frac{t_q + 1}{2}\right), \quad (17)$$

with Legendre nodes t_q and weights w_q on $[-1, 1]$. Note that the interval $[F_k(x_s), F_k(x_e)]$ depends on the pixel column k , so the nodes are placed per pixel.

Cost. For the u form, F_k^{-1} is evaluated at Q nodes for each of the n column-parameters (Qn calls of `betaincinv`), then G_l is evaluated at each resulting y for each of the n row-parameters (Qn^2 calls of `betainc`). An edge therefore costs $O(Qn^2)$ Beta CDF evaluations, and the total cost is $O(E \cdot Q \cdot n^2)$ for E distinct edges.

1.6. Interpolation in the pixel median

With $n = \text{PIXELS}$ large, $O(Qn^2)$ per edge is still costly. But the cell probabilities are **smooth** functions of the medians (m_x, m_y) : (a, b) depends smoothly on m , and the integrals Equation 8 depend smoothly on (a, b) . (The **winner** of a voting method jumps between pixels, but the probabilities do not.) So they are computed exactly only on an $N \times N$ grid of medians (`NODES`, default $N = 49$) and interpolated to all $n \times n$ pixels. If $N \geq n$ no interpolation is used.

1.6.1. Nodes (`node_medians`)

The interpolation variable is $z = \text{logit}(m) = \log(m/(1 - m))$. It stretches the regions near 0 and 1, where (a, b) change fastest. The nodes are the Chebyshev–Lobatto points in z spanning the first and last pixel median:

$$t_j = -\cos\left(\pi \frac{j}{N-1}\right), \quad z_j = Zt_j, \quad m_j = \text{expit}(z_j), \quad j = 0, \dots, N-1, \quad (18)$$

where $Z = \text{logit}(1 - \frac{1}{2n})$. (The code writes t_j as $\sin(\pi \frac{2j-(N-1)}{2(N-1)})$, which is the same thing.) Chebyshev–Lobatto points cluster at the ends of the interval, which further concentrates nodes near the edges of the square and avoids the Runge phenomenon.

1.6.2. Barycentric interpolation (`_interpolation_matrix`)

For Chebyshev–Lobatto nodes the barycentric formula of the second kind is

$$p(z) = \frac{\sum_j \frac{\lambda_j}{z - z_j} h_j}{\sum_j \frac{\lambda_j}{z - z_j}}, \quad \lambda_j = (-1)^j, \quad \lambda_0, \lambda_{N-1} \text{ halved}, \quad (19)$$

where h_j are the values at the nodes. (The weights are invariant under the affine map $t \rightarrow Zt$, up to a common factor that cancels.) When a target coincides with a node, the node value is used directly. Evaluated at all n pixel medians this is a fixed $n \times N$ matrix L , and on the 2-D tensor grid

$$P_{\text{pixels}}[\cdot, \cdot, r] = L P_{\text{nodes}}[\cdot, \cdot, r] L^T, \quad (20)$$

applied as two matrix products (`interpolate_to_pixels`). The final product is carried out in `float32` because the result is by far the largest array; the rounding ($\approx 10^{-7}$) can only affect pixels whose winning margin is equally small. Values are finally clipped to $[0, 1]$.

Because the probabilities are analytic in z , the interpolation error decreases exponentially in N . The interpolation reproduces constants exactly, so the probabilities of each pixel still sum to one up to rounding.

Only the node probabilities are cached on disk (`generate_ranking_probabilities`), together with the medians and parameters; interpolation happens on every load.

1.7. Validation (`monte_carlo_check`)

For selected pixels, M voters (default $2 \cdot 10^6$) are sampled from the pixel’s Beta $\times$ Beta distribution, each is ranked directly by sorting distances, and the empirical ranking frequencies are compared with the exact probabilities. The maximum absolute difference should be of the order of the Monte Carlo error $\sqrt{p(1-p)/M} \leq \frac{1}{2\sqrt{M}} \approx 3.5 \cdot 10^{-4}$.

1.8. Summary of the pipeline

1. For each node median solve Equation 3–Equation 6 for (a, b) (with mirror symmetry and continuation).
2. Build the arrangement of the $\binom{C}{2}$ bisectors in the unit square; each cell has one ranking.
3. For every distinct cell edge, compute $\int \omega$ for all pairs of node parameters (closed form, $u = F(x)$ or $v = G(y)$ substitution, Gauss–Legendre quadrature).
4. Sum signed edge integrals into cell probabilities (Green’s theorem).
5. Interpolate from the $N \times N$ nodes to the $n \times n$ pixels with barycentric Chebyshev interpolation in $\text{logit}(m)$.

1.9. Normal voters (normal.py)

normal.py computes the same ranking probabilities for the original Yee model, in which the voters of a pixel are normally distributed. Chapter 2 compares the two models; this section only describes the computation.

1.9.1. Model

The voters of the pixel with centre $m = ((k + \frac{1}{2})/n, (l + \frac{1}{2})/n)$ are

$$(X, Y) \sim \mathcal{N}(m, \sigma^2 I), \quad (21)$$

isotropic and **not** restricted to the unit square. To make the two models comparable, σ is set from the same constant d (DEVIATION): for $X \sim \mathcal{N}(m_x, \sigma^2)$, $E|X - m_x| = \sigma\sqrt{2/\pi}$, so

$$\sigma = d\sqrt{\frac{\pi}{2}} \quad (d = 0.3 \Rightarrow \sigma \approx 0.376), \quad (22)$$

(sigma_from_deviation). Both models then have the same mean absolute deviation from the median in each coordinate; for the normal the median is also the mean.

1.9.2. Cells in a larger box (normal_cells)

Voters can lie outside the square, so the bisector arrangement is built in the box $[-L, 1 + L]^2$ with $L = 10\sigma$ (BOX). For every pixel centre in $[0, 1]^2$ the mass outside the box is at most $4\Phi(-10) \approx 3 \cdot 10^{-23}$. The similarity map $p \mapsto (p + L)/(1 + 2L)$ maps perpendicular bisectors to perpendicular bisectors and preserves the order of distances, so ranking_cells is applied to the mapped candidates and the polygons are mapped back.

1.9.3. Exact cell probabilities with Owen’s T function

No quadrature is needed. For a convex CCW polygon K with vertices $p_1, \dots, p_r$ and any point m , the indicator of K is, almost everywhere, the signed sum of the indicators of the triangles (m, p_k, p_{k+1}) (a fan from m ; triangles oriented clockwise count negatively). Hence

$$P(K) = \sum_{\text{edges } s \rightarrow e} P_{\pm}(m, s, e). \quad (23)$$

Each triangle (m, s, e) is split at the foot q of the perpendicular from m onto the edge line into two right triangles with legs d (from m to q) and t (from q along the line). The isotropic normal is invariant under rotations, so in units of σ a right triangle is $\{0 \leq z_1 \leq h, 0 \leq z_2 \leq az_1\}$ with $h = d/\sigma$, $a = t/d$, for standard normal (Z_1, Z_2) . The wedge $\{z_1 > 0, 0 < z_2 < az_1\}$ holds $\arctan(a)/(2\pi)$ of the mass, and the part of it beyond $z_1 = h$ is by definition Owen’s T function,

$$T(h, a) = \frac{1}{2\pi} \int_0^a \frac{\exp(-h^2(1+x^2)/2)}{1+x^2} dx = P(Z_1 > h, 0 < Z_2 < aZ_1) \quad (h, a \geq 0). \quad (24)$$

Therefore the right triangle has probability

$$R(d, t) = \frac{\arctan(t/d)}{2\pi} - T(d/\sigma, t/d). \quad (25)$$

Both terms are odd in t ; T is even in h and odd in a , so Equation 25 with a **signed** d is the signed probability. For the edge $s \rightarrow e$ with unit direction u and left normal $u^\perp = (-u_y, u_x)$ let

$$d = (m - s) \cdot u^\perp, \quad t_s = (s - m) \cdot u, \quad t_e = t_s + |e - s|, \quad (26)$$

where $d > 0$ when m lies left of the edge, i.e. inside a CCW cell. Then

$$P_\pm(m, s, e) = R(d, t_e) - R(d, t_s), \quad (27)$$

and 0 when $d = 0$ (the triangle degenerates). For the $N \times N$ grid of pixel centres d, t_s, t_e are outer sums, so each edge costs two `owens_t` evaluations per pixel; the cells are computed in parallel threads (`owens_t` releases the GIL). Five candidates take about 0.1 s on 49×49 nodes. The pairwise shares match their closed form (Equation 29 in Chapter 2) to 10^{-14} in the tests.

1.9.4. Nodes

Unlike the Beta parameters, nothing changes faster near 0 or 1, so the nodes are Chebyshev–Lobatto points in the median itself, $m_j = \frac{1}{2} + (\frac{1}{2} - \frac{1}{2n})t_j$, and the barycentric interpolation of Section 1.6 is applied in m instead of $\text{logit}(m)$ (`interpolate_to_pixels(..., transform=...)`). The logit nodes would be far too sparse in the middle for small σ (maximum error at 120 pixels, $N = 49$):

nodes in	$\sigma = 0.376$	$\sigma = 0.125$	$\sigma = 0.063$
m	$1.7 \cdot 10^{-7}$	$3.6 \cdot 10^{-7}$	$2.9 \cdot 10^{-7}$
$\text{logit}(m)$	$1.6 \cdot 10^{-7}$	$7.5 \cdot 10^{-5}$	$1.4 \cdot 10^{-2}$

The floor of about 10^{-7} is the `float32` rounding of the result.

2. Beta versus normal voters

The original Yee diagrams put normally distributed voters around every point; this project replaced them by Beta distributed voters (Chapter 1). With the same candidates the two models give visibly different diagrams, and the difference is largest where one would expect it least: for Condorcet methods. On votingmethods.net/yee (normal voters) the River method splits the square into one clean region per candidate, while Schulze on the Beta model has distorted regions and even pixels without a Condorcet winner (Figure 2). This chapter explains why.

All numbers below use the candidates A–E of `const.py` (Figure 1), $d = 0.3$ (DEVIATION), 300×300 pixels and 49×49 nodes; they are produced by `docs/figures.py`.

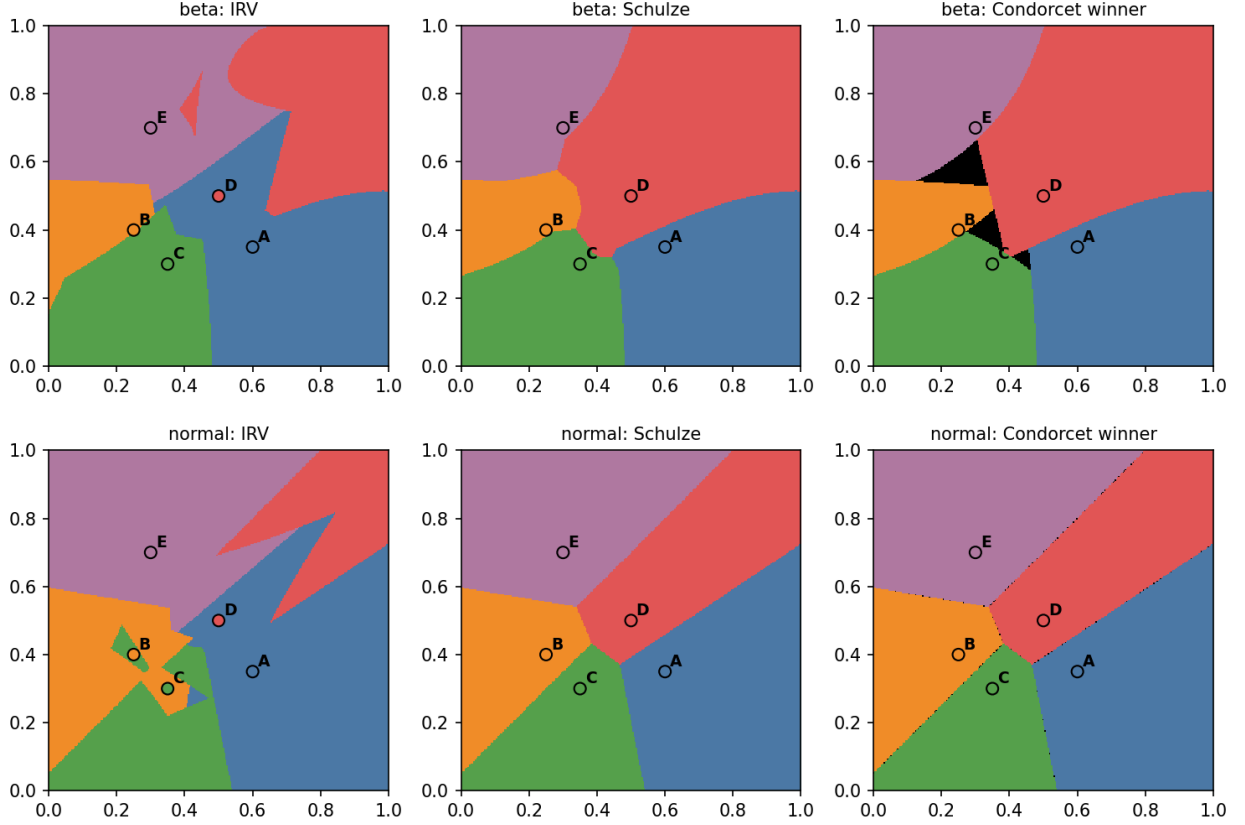


Figure 2: Beta (top) and normal voters (bottom) with the same $d = 0.3$. Normal Schulze and the normal Condorcet winner are exactly the Voronoi diagram of the candidates. Black: no Condorcet winner. The dotted black lines in the bottom right plot are pixels whose centre lies exactly on a bisector (a tie, not a cycle).

2.1. The two models

	Beta (<code>ranking_cells.py</code>)	normal (<code>normal.py</code>)
voters of pixel m	$X \sim \text{Beta}(a_x, b_x)$, $Y \sim \text{Beta}(a_y, b_y)$ independent	$(X, Y) \sim \mathcal{N}(m, \sigma^2 I)$
support	unit square	whole plane
m is	the median of each coordinate	mean and median, centre of symmetry
spread	$E X - m_x = d$	$E X - m_x = d$, i.e. Equation 22
symmetric about m	only for $m = (\frac{1}{2}, \frac{1}{2})$	always

votingmethods.net uses a standard deviation of 0.4 by default ($d \approx 0.319$ here) and samples only 140 voters per pixel, so its diagrams also carry Monte Carlo noise (ragged IRV borders, small islands). The probabilities here are exact.

2.2. Pairwise majorities

By Equation 7 a voter at p prefers c_i to c_j exactly when $p \cdot \nu < \omega$ with $\nu = c_j - c_i$ and $\omega = (|c_j|^2 - |c_i|^2)/2$. The share of the voters of pixel m who prefer c_i to c_j is therefore

$$\pi_{ij}(m) = P_m(X \cdot \nu < \omega), \quad (28)$$

and it depends only on the one-dimensional projection $X \cdot \nu$ of the voters onto the direction from c_i to c_j . Every Condorcet method only looks at these numbers.

2.2.1. Normal voters: the majority is decided by distance

The projection of an isotropic normal is normal, $X \cdot \nu \sim \mathcal{N}(m \cdot \nu, \sigma^2 |\nu|^2)$, so

$$\pi_{ij}(m) = \Phi\left(\delta_{ij} \frac{m}{\sigma}\right), \quad \delta_{ij}(m) = \frac{\omega - m \cdot \nu}{|\nu|}, \quad (29)$$

where $\delta_{ij}(m)$ is the signed distance of m from the bisector, positive on the side of c_i . A majority prefers c_i to c_j if and only if $\delta_{ij}(m) > 0$, that is, if and only if $|m - c_i| < |m - c_j|$. Consequently:

1. The majority relation of every pixel is the order of the candidates by distance from m . It is transitive: **there are no Condorcet cycles**.
2. The Condorcet winner always exists and is the candidate nearest to m , so **every** Condorcet method (Schulze, River, Ranked Pairs, Minimax, ...) draws exactly the Voronoi diagram of the candidates, `methods.ideal`.
3. This holds **for every** σ . The only property used is that every line through m has half of the voters on each side, which is true for every distribution that is centrally symmetric about m .

This is why the River diagram of `votingmethods.net` looks like a Voronoi diagram; its borders agree with the bisectors (e.g. the D|A bisector meets $x = 1$ at $y = 0.725$ and the D|E bisector meets $y = 1$ at $x = 0.8$). The spread σ still matters for methods that use more than pairwise majorities (FPTP, IRV, Borda).

2.2.2. Beta voters: the median is a median only along the axes

For the Beta model m is the median of X and of Y separately. If ν is parallel to an axis, $X \cdot \nu$ is a multiple of one coordinate, its median is $m \cdot \nu$, and the line through m perpendicular to ν splits the voters exactly in half (`test_median_splits_voters_in_half`). For an oblique ν this fails: the median of $\nu_x X + \nu_y Y$ is in general **not** $\nu_x m_x + \nu_y m_y$, because medians are not additive for skewed variables. For every direction ν there is a different “effective centre” of the pixel, shifted from m towards the mean of the voters. Consequently:

1. The Condorcet winner is no longer the candidate nearest to m , so Condorcet regions are distorted.
2. Different pairs of candidates are compared from different effective centres, so the majority relation need not be transitive: **Condorcet cycles appear** (black in Figure 2). If some point were a median in every direction (a “total median”, Davis, DeGroot and Hinich, 1972), the argument of the normal case would apply with that point in place of m ; a product of skewed Beta marginals generally has none.

2.2.3. How skewed the Beta is at $d = 0.3$

The uniform distribution on $[0, 1]$ has $E|X - \frac{1}{2}| = \frac{1}{4}$, and $\text{Beta}(\frac{1}{2}, \frac{1}{2})$ has $1/\pi \approx 0.318$. So $d = 0.3$ is more spread than uniform and the Beta is U-shaped ($a, b < 1$, mass piled at both ends) **even at the centre**:

median	a	b	mean	$P(X < 0.1)$	$P(X > 0.9)$
0.50	0.602	0.602	0.500	0.18	0.18
0.70	0.619	0.393	0.612	0.12	0.32
0.90	0.380	0.175	0.685	0.15	0.50
0.98	0.215	0.093	0.698	0.19	0.58

Near the edges this is forced by the constraints. With median m close to 1 the upper half of the voters lies in $[m, 1]$ and contributes at most $(1 - m)/2$ to $E|X - m|$, so the lower half has to supply the rest:

$$E[m - X \mid X < m] \geq 2d - (1 - m), \quad (30)$$

which for $m = 0.98$ means that the lower half of the voters lies on average at least 0.58 below the median. Moving the median towards the edge **pushes the other half of the voters further away** (Section 2.3 uses this twice).

2.2.4. Example: the pull towards the centre

The pixels below lie on A's side of the bisector of $A = (0.6, 0.35)$ and the centre candidate $D = (0.5, 0.5)$; δ is their distance from it. Share of voters preferring A:

pixel m	δ	normal	Beta	Beta mean
(0.95, 0.55)	0.118	0.623	0.477	(0.694, 0.530)
(0.90, 0.60)	0.049	0.551	0.441	(0.685, 0.559)
(0.80, 0.45)	0.118	0.623	0.523	(0.655, 0.470)

In the first two pixels the median voter is closer to A, yet D wins the pairwise contest: the long tails of the Beta point towards the middle of the square, and so does the effective centre.

2.2.5. The whole diagram

Leaving out the 357 pixels whose centre lies exactly on a bisector (where both candidates are correct winners):

	Schulze $\neq$ Voronoi	no Condorcet winner	area of D (Voronoi: 0.208)
Beta	18 125 pixels (20.2 %)	1 844 pixels (2.1 %)	0.351
normal	0	0	0.208

The centre candidate D gains most: the Beta electorates of the whole upper right part of the square are pulled towards the centre, where D is.

2.3. Round edges

Under normal voters the borders of Condorcet methods are bisectors, i.e. straight lines. The borders of FPTP and IRV are curved in both models, and the Beta model adds bends and hooks near the edges of the square. The key is to look at a first-choice share as a function of the pixel: it is the Voronoi cell of the candidate, **blurred**.

2.3.1. First-choice shares are blurred Voronoi cells

The FPTP winner changes where two first-choice shares are equal,

$$s_i(m) = s_j(m), \quad s_i(m) = P_m(X \in V_i) = \int_{V_i} f_m(p) dp, \quad (31)$$

with V_i the Voronoi cell of c_i and f_m the voter density of pixel m . For normal voters $f_m(p) = \varphi_\sigma(p - m)$ is one Gaussian bump moved to m , and since it is symmetric,

$$s_i = \mathbb{1}_{V_i} * \varphi_\sigma, \quad (32)$$

the indicator of the cell (1 inside, 0 outside) convolved with a Gaussian of radius σ : the cell **blurred** like an out-of-focus photograph. The FPTP diagram is obtained by blurring every Voronoi cell and colouring each pixel by the brightest one; for $\sigma \rightarrow 0$ this is the Voronoi diagram itself. IRV does the same once per round, with the cells V_i^S of the candidates S still in the race. A border is therefore a **contour line** where two blurred cells are equally bright, and the question is when such a contour line is straight.

For Beta voters there is no single bump: f_m changes its shape with m (U-shaped, piled against the walls, Figure 6 and Figure 8), so the blur itself varies across the square. This is the source of the bends near the edges below.

2.3.2. Straight versus round: a proof by blurring a corner

Half-plane: straight. With two candidates each cell is a half-plane, and by Equation 29 its blurred version is $\Phi(\delta/\sigma)$, a function of the distance δ from the bisector only. All its contour lines are parallel to the bisector, and the tie $s_i = s_j = \frac{1}{2}$ is the bisector itself.

Corner: round. Let a cell be the quadrant $Q = \{x < \frac{1}{2}, y < \frac{1}{2}\}$. The two coordinates of the normal are independent, so its blurred version is exactly

$$s_Q(m) = \Phi\left(\frac{\frac{1}{2} - m_x}{\sigma}\right) \Phi\left(\frac{\frac{1}{2} - m_y}{\sigma}\right). \quad (33)$$

Far from the corner one factor is 1 and the contour lines are straight lines parallel to the edges of Q . At the corner they bend round (Figure 3, middle). For example the contour $s_Q = \frac{1}{2}$ approaches the edges of Q far away, but on the diagonal it needs $\Phi(u)^2 = \frac{1}{2}$, $u = \Phi^{-1}(1/\sqrt{2}) = 0.545$: it passes $0.545\sqrt{2}\sigma \approx 0.77\sigma$ **inside** the corner. Blurring rounds the corner with a radius of the order of σ , exactly as blurring a photograph of a square gives a rounded square.

Vertices: the widest cell wins. A cell that is a wedge with angle α at a Voronoi vertex receives exactly $\alpha/(2\pi)$ of the voters of the pixel centred at that vertex (the Gaussian is rotationally symmetric). Where several cells meet, the one with the largest angle wins at the vertex, and the fronts of the others recede by a distance of the order of σ along round contour lines. Far from the vertices (distance $\gg \sigma$) only two cells are nearby and the border returns to their bisector.

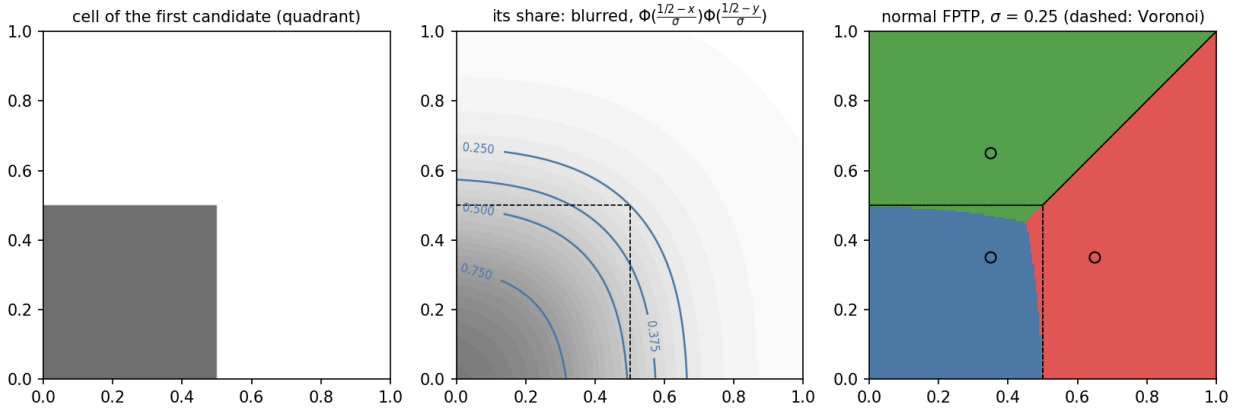


Figure 3: Three candidates whose cells meet at $(\frac{1}{2}, \frac{1}{2})$ with angles 90° , 135° , 135° (normal voters, $\sigma = 0.25$). Left: the quadrant cell of the lower-left candidate. Middle: its share Equation 33 with contour lines. Right: FPTP.

At the vertex the shares are $\frac{1}{4}$, $\frac{3}{8}$, $\frac{3}{8}$, so the quadrant candidate loses its corner, along a round front.

2.3.3. Seven candidates

Figure 4 shows FPTP for seven candidates (B removed, three added on the left). With a small blur ($d = 0.1$) the diagram is the Voronoi diagram with rounded corners at the vertices. With $d = 0.3$ the blur is wider than several cells:

- the centre candidate $(0.5, 0.5)$ has a thin band as its cell. Blurred, a thin band stays dim everywhere, so it wins **nowhere**, not even at its own position;
- borders appear between candidates whose cells do not even touch, e.g. $(0.3, 0.7)$ and $(0.6, 0.35)$. Such a border has no bisector to follow at all: it is purely a contour line of two blurred cells, and it is curved.

The Beta diagram shows the same effects, plus the distortions of a blur that changes from pixel to pixel (the bowl-shaped border under $(0.3, 0.7)$).

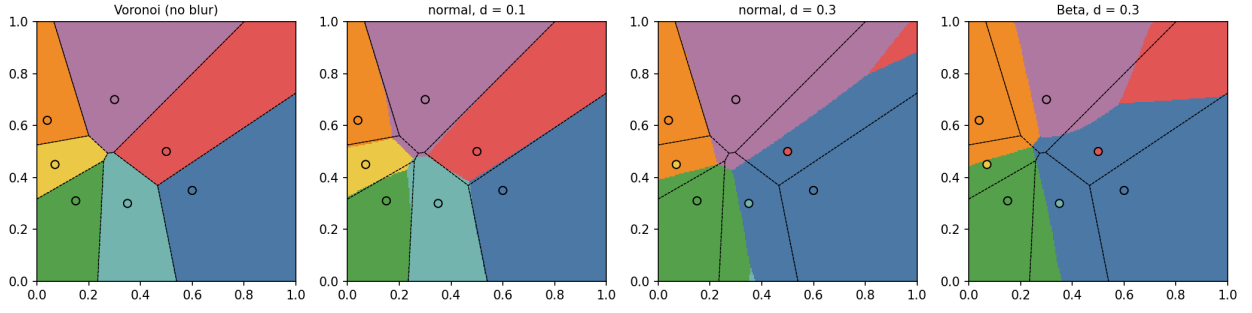


Figure 4: FPTP for seven candidates; dashed: Voronoi borders. From left: no blur (Voronoi), normal voters with $d = 0.1$ and $d = 0.3$, Beta voters with $d = 0.3$.

2.3.4. The round IRV edge

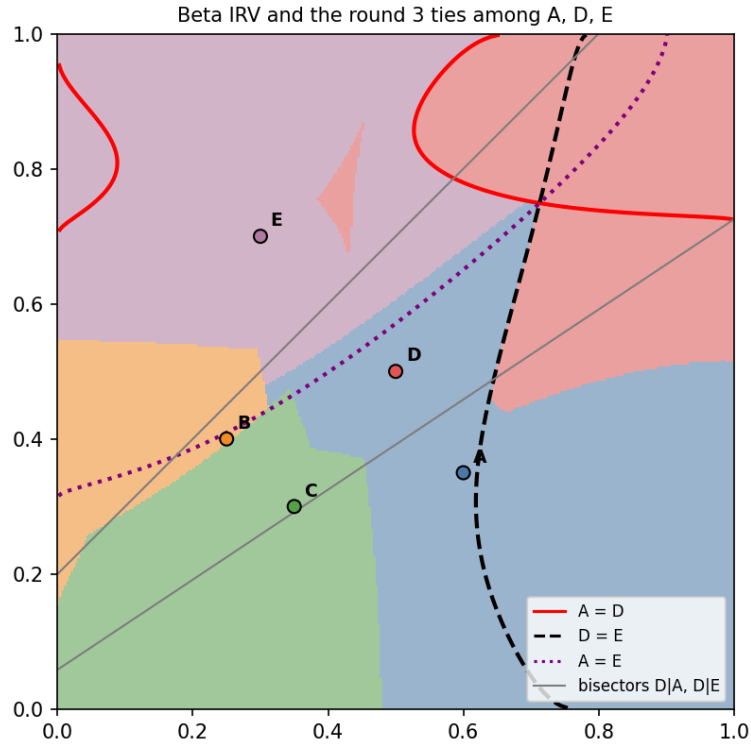


Figure 5: Beta IRV (faded) with the three round 3 tie curves among A, D, E. The red curve $s_A = s_D$ is the round edge of D's region; all three curves meet at the kink near $(0.71, 0.75)$. Gray: the bisectors $D|A$ and $D|E$.

In the upper right part B and C are eliminated first. In round 3 A, D and E remain, and D's cell $V_D^{\{A,D,E\}}$ is the diagonal band between the bisectors $D|A$ and $D|E$ that widens towards the corner $(1, 1)$; A has the bottom right, E the top left. Whoever of A and D has fewer votes is eliminated:

- D out: the final is A against E, and E wins;
- A out: the final is D against E, and D wins (the pull towards the centre again).

So the edge of D's region is the tie curve $s_A = s_D$ of round 3 (Figure 5). It bends back on itself because along the column $x = 0.55$ the two shares are not monotonic:

y	winner	s_A	s_D	s_E	$P(Y < 0.2)$
0.678	E	0.327	0.284	0.389	0.195
0.738	E	0.303	0.286	0.411	0.185
0.798	E	0.285	0.284	0.431	0.182

0.858	D	0.274	0.279	0.447	0.187
0.918	D	0.270	0.270	0.460	0.199
0.978	E	0.274	0.254	0.472	0.226

- Low in the column many voters are still below the D|A bisector, so A is stronger.
- Going up, voters leave A's cell into D's band: s_A falls, s_D rises.
- Higher up, voters cross the D|E bisector into E's part of the top edge, and s_D falls again, while s_A **rises** again: with the median of Y approaching 1 the fixed deviation pushes the lower half of the voters down (Section 2.2.3), $P(Y < 0.2)$ grows from 0.182 to 0.226, and those voters are in A's cell.

D is ahead of A only in a middle band, so the tie curve closes around it. At the kink the three shares are equal; below it the edge follows the tie $s_D = s_E$ (dashed).

Figure 6 shows this directly: sampled voters of four pixels of the column, coloured by their first choice among A, D and E. With Beta voters the cloud does not move, it is squeezed against the walls: a dense row forms at the top wall, mostly E (purple), and another at the bottom wall, which is A's (blue). So s_A stops falling and rises again (0.317, 0.285, 0.274, 0.275) while D's share drops (0.285 to 0.253). With normal voters the whole cloud simply moves up: s_A falls and s_E rises monotonically.

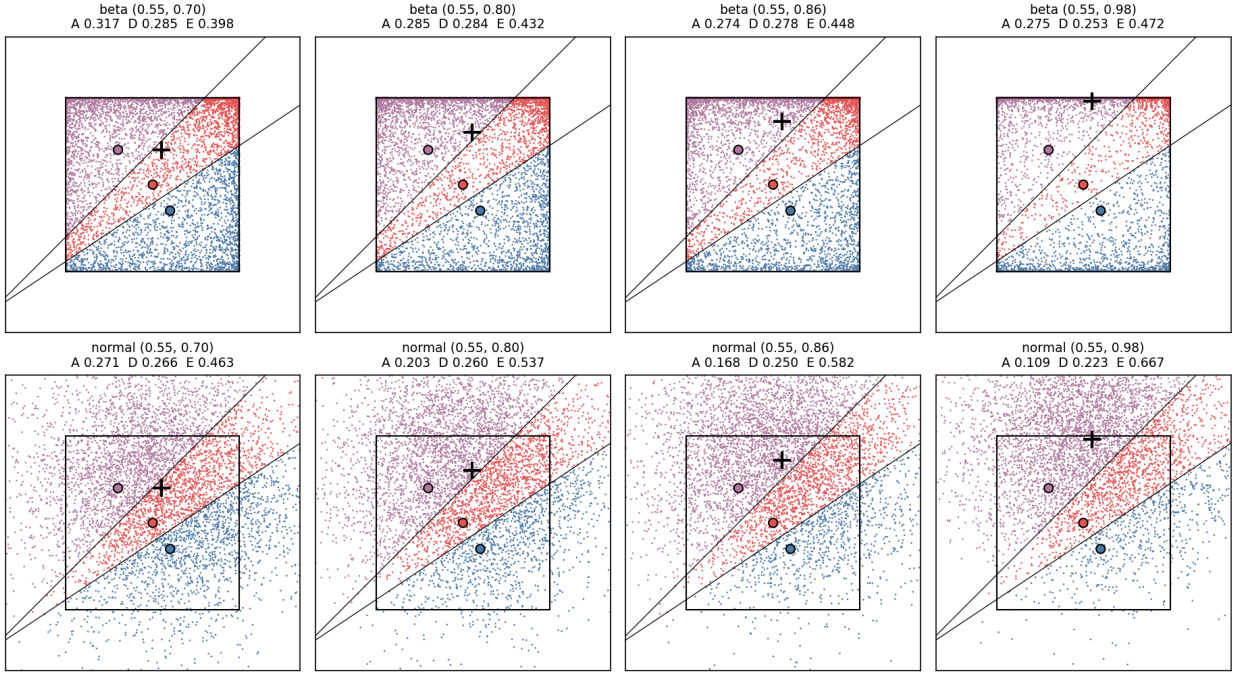


Figure 6: Voters of the pixels $(0.55, y)$ coloured by their first choice among A (blue), D (red) and E (purple), with the bisectors D|A and D|E; +: pixel median (Beta) or mean (normal). Titles: the exact round 3 shares.

With normal voters D also beats A in round 3 around $y \approx 0.74$, but a little higher D loses the final against E, because the pixel is closer to E; the edge there is the straight D|E bisector and the curved round 3 tie is not visible:

y (normal)	winner	s_A	s_D	s_E
0.678	E	0.288	0.266	0.445
0.738	D	0.245	0.265	0.490
0.798	E	0.205	0.260	0.535
0.858	E	0.169	0.251	0.579

2.3.5. Hooks at the edge of the square (FPTP)

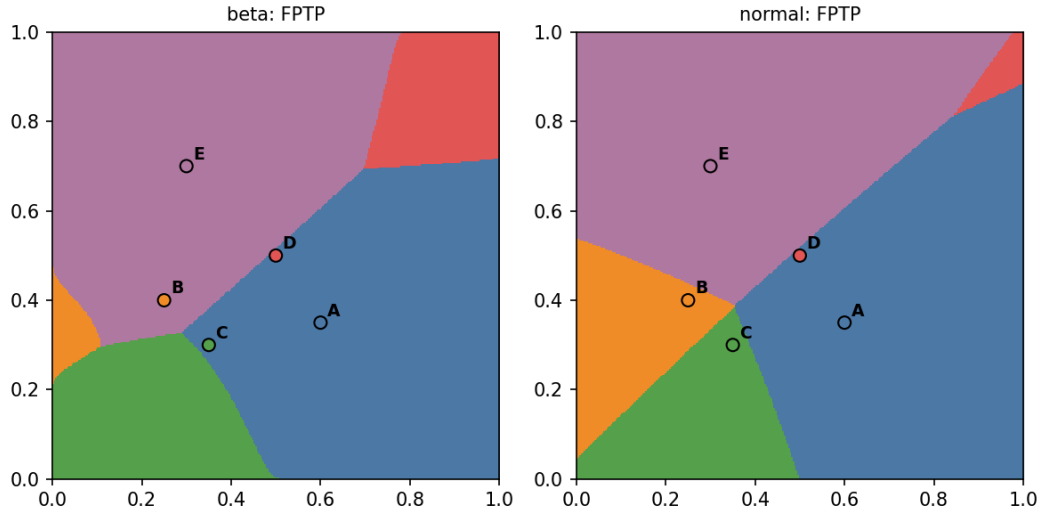


Figure 7: FPTP. With Beta voters the region of B flares out in the last few hundredths before the left edge; with normal voters it widens smoothly.

$B = (0.25, 0.4)$ is the leftmost candidate and its Voronoi cell widens towards the left edge: the bisector $B|C$ ($y = x + 0.05$) falls as x decreases, and at $x = 0$ the cell spans $0.05 < y < 0.596$. The more voters sit close to $x = 0$, the more of them fall into the wide part of B's cell. The Beta squeezes the voters against the wall as the median approaches it — the same effect of the fixed deviation as above:

median x	$P(X < 0.02)$	Beta B E	Beta C B	normal B E	normal C B
0.200	0.259	—	—	0.455	0.238
0.080	0.400	0.362	0.278	0.505	0.125
0.030	0.478	0.422	0.245	0.525	0.075
0.005	0.547	0.458	0.218	0.535	0.048

(heights y of the borders of B's region in the column; “—”: B wins nowhere). With Beta voters B's share jumps within the last few hundredths of the square and the borders hook (B|E bends up, C|B down). With normal voters the widening is smooth: the voters just continue beyond $x = 0$ into the wider part of the cell, and nothing piles up at the wall.

Figure 8 shows why. Between $x = 0.2$ and $x = 0.02$ the Beta voters of the pixel collapse onto the wall, into the wide part of B's cell: B's share grows from 0.22 to 0.29 while C's falls from 0.19 to 0.16, most of it in the last few hundredths. The normal cloud keeps its shape and only moves (B's share grows smoothly, 0.32, 0.40, 0.44), so its borders have no hook.

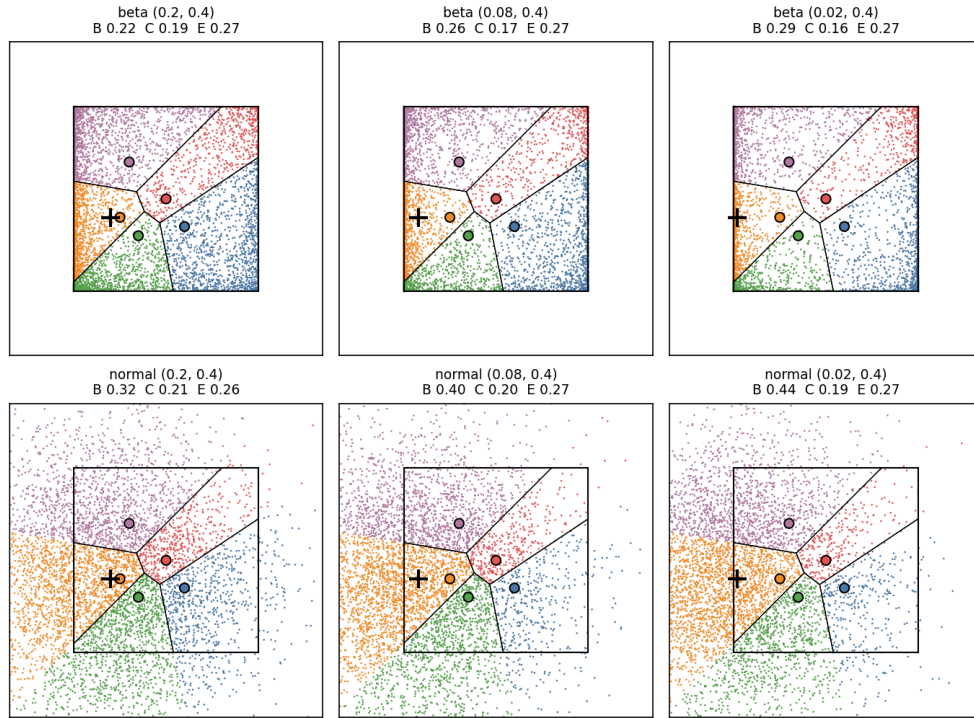


Figure 8: Voters of the pixels $(x, 0.4)$ approaching the left wall, coloured by their first choice (FPTP), with the Voronoi borders; titles: shares of B, C and E.

In short: **curved** FPTP and IRV borders are generic (ties between masses of polygons), while the **hooks and bends near the edges** are a property of the Beta model with a fixed deviation, the same property that distorts the Condorcet diagrams.

2.4. Summary

- Normal voters are symmetric about the pixel centre, so pairwise majorities follow distance: no cycles, and every Condorcet method draws the Voronoi diagram, whatever σ .
- Beta voters have the pixel centre as the median of each coordinate only. Majorities between candidates on a diagonal are decided by the skew of the distribution, which pulls the effective centre towards the middle of the square: Condorcet regions are distorted (20 % of the pixels for $d = 0.3$) and cycles appear.
- A first-choice share is the candidate's Voronoi cell blurred by the voter distribution. Blurring keeps half-planes straight but rounds corners (radius of order σ), so FPTP and IRV borders are curved in both models.
- At $d = 0.3$ the Beta is U-shaped everywhere, and near the edges the fixed deviation pushes the other half of the voters away: the blur changes shape across the square. This produces the round IRV edge and the FPTP hooks at the walls of the square.